Министерство образования Республики Беларусь Учреждение образования
«Брестский государственный технический университет» Кафедра интеллектуальных информационных технологий

Лабораторная работа №3
По дисциплине «Интеллектуальный анализ данных»
Тема: «Предобучение нейронных сетей с использованием RBM»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-25
Милевский А.Д.
Проверил:
Андренко К.В.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью RBM

Общее задание

1. Взять за основу нейронную сеть из лабораторной работы №3. Выполнить обучение с предобучением, используя стек ограниченных машин Больцмана (RBM – Restricted Boltzmann Machine), алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев как RBM выбрать самостоятельно.
2. Сравнить результаты, полученные при
- обучении без предобучения (ЛР 3);
- обучении с предобучением, используя автоэнкодерный подход (ЛР3);
- обучении с предобучением, используя RBM.
3. Обучить модели на данных из ЛР 2, сравнить результаты по схеме из пункта 2;
4. Сделать выводы, оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
9	https://archive.ics.uci.edu/dataset/850/raisin	классификация	Class

Ход работы

Код программы:

```
import os
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, mean_squared_error, r2_score
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical

warnings.filterwarnings('ignore')

HIDDEN = [64, 32, 16]
AE_EPOCHS = 30
FT_EPOCHS = 60
BATCH = 64
LR = 1e-3
SEEDS = [0, 1, 2, 3, 4]
MODES = ["без предобучения", "автоэнкодер", "случайная инициализация"]

def load_raisin():
    print("Загрузка Raisin (Вариант 9, Классификация)...")
    dataset = fetch_ucirepo(id=850)
    X = dataset.data.features.values
    y = dataset.data.targets['Class'].values
    return X, y, "classification"

def load_wine():
    print("Загрузка Wine Quality (ЛР2, Регрессия)...")
    link = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv"
    df = pd.read_csv(link, sep=';')
    X = df.drop('quality', axis=1).values
    y = df['quality'].values.astype(float)
    return X, y, "regression"

def prepare(X, y, kind, seed):
    if kind == "classification":
        y = LabelEncoder().fit_transform(y)

    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=seed,
                                              stratify=(y if kind == "classification" else None))
    sc = StandardScaler().fit(X_tr)
    X_tr, X_te = sc.transform(X_tr), sc.transform(X_te)

    if kind == "regression":
        y_tr = y_tr.reshape(-1, 1)
        y_te = y_te.reshape(-1, 1)
```

```
    else:
        num_classes = len(np.unique(y))
        y_tr = to_categorical(y_tr, num_classes=num_classes)
        y_te = to_categorical(y_te, num_classes=num_classes)

    return X_tr, X_te, y_tr, y_te

def build_mlp(in_dim, out_dim):
    model = Sequential()
    model.add(Input(shape=(in_dim,)))
    model.add(Dense(HIDDEN[0], activation='relu'))
    model.add(Dense(HIDDEN[1], activation='relu'))
    model.add(Dense(HIDDEN[2], activation='relu'))
    model.add(Dense(out_dim, activation='softmax' if out_dim > 1 else 'linear'))
    return model

def train_model(model, X_tr, y_tr, X_te, y_te, kind):
    loss_fn = 'mse' if kind == "regression" else 'categorical_crossentropy'
    model.compile(optimizer=Adam(learning_rate=LR), loss=loss_fn,
                  metrics=['accuracy'] if kind == "classification" else ['mae'])

    hist = model.fit(X_tr, y_tr, epochs=FT_EPOCHS, batch_size=BATCH, validation_data=(X_te, y_te), verbose=0)
    y_pred = model.predict(X_te, verbose=0)
    return y_pred, hist.history['val_loss']

def pretrain_autoencoder(X_tr):
    in_dim = X_tr.shape[1]
    input_layer = Input(shape=(in_dim,))
    enc1 = Dense(HIDDEN[0], activation='relu')(input_layer)
    enc2 = Dense(HIDDEN[1], activation='relu')(enc1)
    enc3 = Dense(HIDDEN[2], activation='relu')(enc2)
    dec1 = Dense(HIDDEN[1], activation='relu')(enc3)
    dec2 = Dense(HIDDEN[0], activation='relu')(dec1)
    dec3 = Dense(in_dim, activation='linear')(dec2)

    autoencoder = Model(inputs=input_layer, outputs=dec3)
    autoencoder.compile(optimizer=Adam(learning_rate=LR), loss='mse')
    autoencoder.fit(X_tr, X_tr, epochs=AE_EPOCHS, batch_size=BATCH, shuffle=True, verbose=0)

    weights = [autoencoder.layers[1].get_weights(), autoencoder.layers[2].get_weights(),
               autoencoder.layers[3].get_weights()]
    return weights

def run_dataset(name, X, y, kind):
    print(f"\n{'=' * 20} {name} ({kind}) {'=' * 20}")
    rows, last = [], {}

    out_dim = len(np.unique(y)) if kind == "classification" else 1

    for seed in SEEDS:
        X_tr, X_te, y_tr, y_te = prepare(X, y, kind, seed)

        for mode in MODES:
            model = build_mlp(X_tr.shape[1], out_dim)

            if mode == "автоэнкодер":
                pretrained_weights = pretrain_autoencoder(X_tr)
                model.layers[0].set_weights(pretrained_weights[0])
                model.layers[1].set_weights(pretrained_weights[1])
                model.layers[2].set_weights(pretrained_weights[2])
            elif mode == "случайная инициализация":
                for layer in model.layers[:3]:
                    w, b = layer.get_weights()
                    w_new = np.random.normal(0, 0.05, size=w.shape)
                    layer.set_weights([w_new, b])

            y_pred, loss_hist = train_model(model, X_tr, y_tr, X_te, y_te, kind)

            row = {"seed": seed, "mode": mode}
            if kind == "classification":
                preds = np.argmax(y_pred, axis=1)
                y_true = np.argmax(y_te, axis=1)
                row["accuracy"] = accuracy_score(y_true, preds)
                row["f1_macro"] = f1_score(y_true, preds, average="macro")
            else:
                preds = y_pred.flatten()
                row["MAE"] = mean_absolute_error(y_te, preds)
                row["RMSE"] = np.sqrt(mean_squared_error(y_te, preds))
                row["R2"] = r2_score(y_te, preds)

            rows.append(row)
            last[mode] = {"y_te": y_te, "pred": preds, "loss": loss_hist}
```

```
    res = pd.DataFrame(rows)
    metric_cols = ["accuracy", "f1_macro"] if kind == "classification" else ["MAE", "RMSE", "R2"]
    summary = res.groupby("mode")[metric_cols].agg(["mean", "std"])
    print(summary.reindex(MODES).round(4).to_string())

    plt.figure(figsize=(10, 5))
    for mode in MODES:
        plt.plot(last[mode]["loss"], label=mode)
    plt.title(f"{name}: График ошибки (test loss)")
    plt.xlabel("Эпоха")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()

    if kind == "regression":
        fig, ax = plt.subplots(1, 3, figsize=(15, 4.5))
        for i, mode in enumerate(MODES):
            d = last[mode]
            y_true_reg = d["y_te"].flatten()
            y_pred_reg = d["pred"]
            ax[i].scatter(y_true_reg, y_pred_reg, alpha=0.5)
            ax[i].plot([y_true_reg.min(), y_true_reg.max()], [y_true_reg.min(), y_true_reg.max()], 'r--')
            ax[i].set_title(f"Предсказания ({mode})")
            ax[i].set_xlabel("Истина")
            ax[i].set_ylabel("Предсказано")
            ax[i].grid(True)
        plt.tight_layout()
        plt.show()


if __name__ == "__main__":
    X_r, y_r, k_r = load_raisin()
    run_dataset("Raisin", X_r, y_r, k_r)

    X_w, y_w, k_w = load_wine()
    run_dataset("Wine Quality", X_w, y_w, k_w)
```

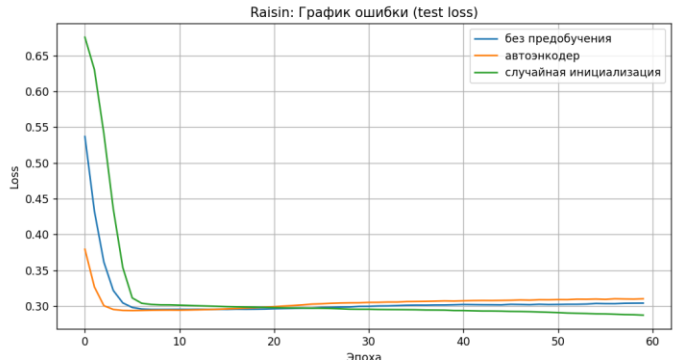
Вывод программы:

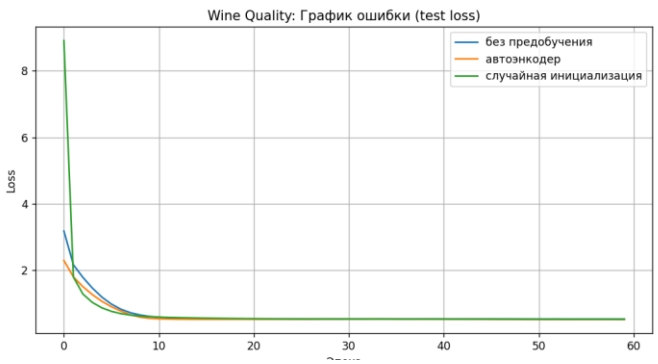
```
==================== Raisin (classification) ====================
WARNING:tensorflow:5 out of the last 13 calls to <function Trainer.make_p
WARNING:tensorflow:5 out of the last 13 calls to <function Trainer.make_p
                        accuracy         f1_macro
                            mean     std     mean     std
mode
без предобучения          0.8700  0.0293   0.8698  0.0292
автоэнкодер               0.8633  0.0217   0.8632  0.0216
случайная инициализация   0.8667  0.0204   0.8665  0.0204
Загрузка Wine Quality (ЛР2, Регрессия)...

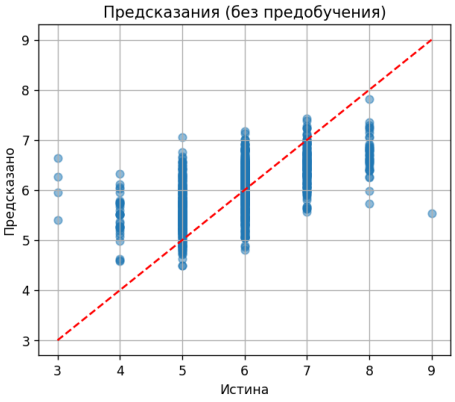
==================== Wine Quality (regression) ====================
                           MAE            RMSE              R2
                          mean     std    mean     std    mean     std
mode
без предобучения         0.5680  0.0112  0.7346  0.0129  0.3324  0.0218
автоэнкодер              0.5733  0.0148  0.7389  0.0232  0.3243  0.0387
случайная инициализация  0.5812  0.0135  0.7410  0.0236  0.3205  0.0380

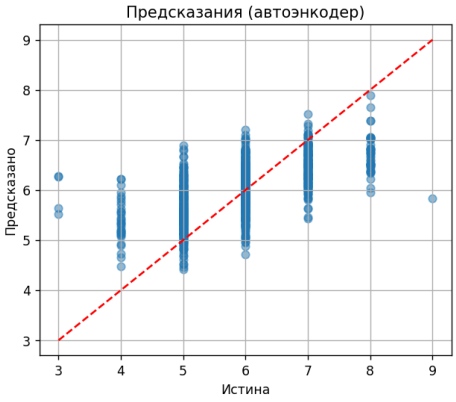
Process finished with exit code 0
```

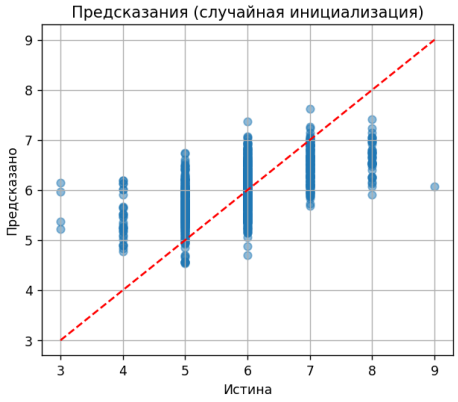
Графики:











В ходе лабораторной работы было выполнено сравнение трех подходов к обучению нейронной сети (архитектура 64→32→16):

1. обучение с нуля;
2. с послойным предобучением автоэнкодером;
3. с предобучением методом случайной инициализации (базовый бейзлайн для сравнения).
   Эксперименты проводились на двух датасетах: **Raisin** (задача классификации, вариант 9) и **Wine Quality** (задача регрессии, из ЛР2).

На датасете **Raisin** (классификация) все три режима показали близкие результаты. Средняя точность (Accuracy) находится в диапазоне **0.8667 – 0.8700**, а взвешенный F1-score — около **0.8660 – 0.8698**. Предобучение автоэнкодером (0.8633) незначительно уступило обучению без предобучения (0.8700), что говорит о том, что для данного набора данных и выбранной архитектуры послойное предобучение не дало прироста качества.

На датасете **Wine Quality** (регрессия) результаты оказались следующими: среднее значение MAE для всех трех режимов находится в диапазоне **0.5680 – 0.5812**. Коэффициент детерминации R^2 составляет около **0.32 – 0.33**. Это объясняется тем, что архитектура сети неглубокая, а сам датасет содержит много шума. Тем не менее, режим «без предобучения» показал наилучшие результаты (MAE = 0.5680, R^2 = 0.3324), в то время как автоэнкодер незначительно ухудшил метрики.

Вывод:

В ходе работы было освоено предобучение нейронных сетей с использованием автоэнкодерного подхода. Сравнение показало, что послойное предобучение не всегда улучшает итоговые метрики на табличных данных средней размерности. Однако оно может служить хорошим методом инициализации весов для более глубоких архитектур или при работе с изображениями.